

Sprint 14 Structured Notes: Testing Foundations, Unit Tests, React Testing Library, TDD/BDD, Assertions, Mocks, Stubs, and Fakes

0. Where You Are Before Sprint 14

Before Sprint 14, you should already understand these React ideas:

- React builds user interfaces using components.
- JSX lets you write UI-like syntax inside JavaScript.
- Props pass data from parent components to child components.
- `children` lets one component wrap other JSX.
- Conditional rendering lets React show different UI based on data.
- `map()` renders lists.
- Stable keys help React track list items.
- Events let React respond to clicks, typing, and form submissions.
- Controlled inputs store form values in React state.
- `useState` lets React remember changing data.
- State updates should be immutable.
- `useReducer` can organize related state actions.
- `useRef` can access DOM elements or store values that should not re-render the UI.
- `useEffect` runs side effects after React renders.
- Cleanup functions stop old timers, listeners, and subscriptions.
- Custom hooks let you reuse hook logic.
- Context API lets you share app-level values like user, theme, or language.
- Data fetching lets React communicate with APIs or backend services.
- Loading, data, and error states make async UI clear.
- `useMemo`, `useCallback`, and `React.memo` help reduce unnecessary work in some cases.
- React Router connects URLs to React components.
- CSS frameworks and preprocessors help organize styling.
- Browser rendering turns HTML, CSS, JavaScript, images, and fonts into pixels.
- Performance tools help you measure before optimizing.

So far, you have mainly learned this question:

How do I build a React app that works?

Sprint 14 starts a new question:

How do I check that my React app still works after I change it?

That is the purpose of testing.

1. Sprint 14 Goal

The goal of Sprint 14 is to understand testing foundations.

By the end of this sprint, you should be able to:

- Explain why testing matters.
- Explain manual testing.
- Explain automated testing.
- Explain the Software Development Life Cycle, or SDLC.
- Write beginner unit tests.
- Understand basic test syntax.
- Understand `test()`, `describe()`, `expect()`, and matchers.
- Understand passing and failing tests.
- Write tests for small utility functions.
- Understand assertions.
- Understand TDD, or Test-Driven Development.
- Understand BDD, or Behavior-Driven Development.
- Understand mocks, stubs, and fakes.
- Understand React Testing Library at a beginner level.
- Test visible component behavior.
- Avoid testing implementation details too early.

Simple version:

Sprint 14 teaches you how to write code that checks whether your other code behaves correctly.

2. Big Mental Model

When you build an app, bugs can appear when you:

- Add a new feature.
- Change old code.
- Refactor a component.
- Update a dependency.
- Fix one bug but accidentally create another bug.

Testing gives you a safety check.

Basic testing flow:

Code behavior

↓

```
Expected result
↓
Test checks the result
↓
Pass = behavior works
Fail = something is wrong
```

Testing is not about proving that your code is perfect.

Testing is about protecting important behavior.

Beginner rule:

If a function or component has important behavior, write a small test for that behavior.

3. What Is Testing?

Testing means checking whether code behaves the way you expect.

Example:

```
expect(2 + 2).toBe(4);
```

This means:

I expect `2 + 2` to be `4`.

If the result is `4`, the test passes.

If the result is not `4`, the test fails.

Simple definition:

Testing is checking expected behavior.

4. Why Testing Matters

Without tests, you usually check your app manually.

Example:

1. Open the browser.
2. Click a button.
3. Type in a form.
4. Submit the form.
5. Check whether the screen looks correct.

This is useful, but it becomes slow as the app grows.

Testing matters because it helps you:

- Find bugs earlier.
- Avoid breaking old features.
- Refactor with more confidence.
- Document how code should behave.
- Check edge cases that are easy to forget manually.
- Save time when repeatedly checking the same behavior.

Beginner rule:

A test is like a small safety alarm for your code.

5. Manual Testing

Manual testing means a human checks the app by using it.

Example:

```
Open app
↓
Click Login
↓
Type email
↓
Type password
↓
Submit form
↓
Check dashboard appears
```

Manual testing is still useful.

It helps you check:

- Visual layout.
- Real browser behavior.
- Full user experience.
- Things that are hard to automate.

But manual testing has problems.

Problem	Simple Meaning	Example
Slow	You must repeat the same clicks	Testing login every day by hand
Easy to forget	You may skip old features	Forgetting to test password reset
Inconsistent	You may test differently each time	Typing different inputs each time
Weak for hidden logic	Some bugs are not visible immediately	A cart total function calculates wrong

Beginner rule:

Manual testing is useful, but manual testing alone is not enough for larger apps.

6. Automated Testing

Automated testing means code checks your app for you.

Example:

```
expect(formatName("react")).toBe("React");
```

This checks whether `formatName("react")` returns `"React"`.

If it does, the test passes.

If it does not, the test fails.

Automated tests are useful because they are:

- Repeatable.
- Faster than checking everything by hand.
- Good for catching regressions.
- Good for checking small logic.
- Useful before refactoring.

A regression means:

Something that used to work breaks again later.

Beginner rule:

Automated tests help protect old behavior while you add new behavior.

7. Manual Testing vs Automated Testing

Topic	Manual Testing	Automated Testing
Who does it?	A person	Code
Speed	Slower	Faster after setup
Repetition	Tiring	Easy to repeat
Good for	User experience and visual checks	Logic and repeatable behavior
Weakness	Easy to forget cases	Requires writing test code

You usually need both.

Manual testing helps you feel the app.

Automated testing helps you repeatedly check behavior.

8. SDLC: Software Development Life Cycle

SDLC means Software Development Life Cycle.

It describes the common stages of building software.

The main stages are:

1. Planning
2. Design
3. Implementation
4. Testing
5. Deployment
6. Maintenance

8.1 Planning

Planning means deciding what you need to build.

Example:

The app needs a signup form.

8.2 Design

Design means deciding how the feature should work and look.

Example:

The signup form needs name, email, password, and a submit button.

8.3 Implementation

Implementation means writing the code.

Example:

Build `SignupForm.jsx`.

8.4 Testing

Testing means checking whether the code works correctly.

Example:

Test that an error appears when the name field is empty.

8.5 Deployment

Deployment means releasing the app so users can use it.

Example:

Publish the app to production.

8.6 Maintenance

Maintenance means fixing bugs, improving features, and updating the app after release.

Example:

Fix a validation bug and add a new test so the bug does not return.

Beginner rule:

Testing is not only an end step. Good developers test throughout the development process.

9. Testing in a React Workflow

In a React app, testing can happen in different places.

Situation	What to Test
Utility function	Check return value
Button component	Check visible button text
Alert component	Check error message appears
Form component	Check label, input, and validation message
Refactor	Run existing tests before and after changes
Bug fix	Add a test for the bug

Example:

If a cart total was wrong, write a test like this:

```
expect(calculateCartTotal([{ price: 10, quantity: 2 }])).toBe(20);
```

Now, if someone breaks the cart total later, the test can fail and warn you.

10. Unit Testing

A unit test checks one small piece of code in isolation.

A unit can be:

- One function.
- One small component.
- One small module.

Simple definition:

A unit test checks one small thing.

Example function:

```
export function add(a, b) {  
  return a + b;  
}
```

Example test:

```
import { add } from "./add";  
  
test("adds two numbers", () => {  
  expect(add(2, 3)).toBe(5);  
});
```

Read it like this:

When I call `add(2, 3)`, I expect the result to be `5`.

Beginner rule:

Start testing small pure functions before testing complex UI.

11. Pure Functions Are Good First Test Targets

A pure function is a function that:

- Takes input.
- Returns output.
- Does not change outside data.
- Gives the same output for the same input.

Example:

```
export function double(number) {  
  return number * 2;  
}
```

This is easy to test:

```
test("doubles a number", () => {  
  expect(double(4)).toBe(8);  
});
```

Why pure functions are beginner-friendly for testing:

- They are small.
- They do not need the browser.
- They do not need React rendering.
- They are predictable.

Good first test targets:

- `formatName()`
- `calculateCartTotal()`
- `getPostTitles()`
- `filterCompletedTasks()`
- `isValidEmail()`

12. Basic Test Syntax

Most beginner tests use this shape:

```
test("description of behavior", () => {  
  expect(actualValue).toBe(expectedValue);  
});
```

Example:

```
test("capitalizes the first letter", () => {  
  expect(formatName("react")).toBe("React");  
});
```

Important parts:

Code	Meaning
<code>test()</code>	Creates one test case
<code>"capitalizes the first letter"</code>	Describes the behavior

Code	Meaning
<code>() => {}</code>	Function that contains the test code
<code>expect(...)</code>	The actual result you are checking
<code>.toBe(...)</code>	The expected result

Read this line:

```
expect(formatName("react")).toBe("React");
```

As:

I expect the result of `formatName("react")` to be `"React"`.

13. `test()`

`test()` defines one test.

Example:

```
test("adds two numbers", () => {
  expect(2 + 3).toBe(5);
});
```

The first argument is a description.

The second argument is a function that runs the test.

Good test descriptions:

```
test("returns 0 for an empty cart", () => {});
test("shows an error message when name is empty", () => {});
test("renders a submit button", () => {});
```

Bad test descriptions:

```
test("works", () => {});  
test("test 1", () => {});  
test("stuff", () => {});
```

Beginner rule:

Test names should describe behavior clearly.

14. describe()

`describe()` groups related tests.

Example:

```
describe("formatName", () => {  
  test("capitalizes lowercase input", () => {  
    expect(formatName("react")).toBe("React");  
  });  
  
  test("returns empty string for empty input", () => {  
    expect(formatName("")).toBe("");  
  });  
});
```

`describe()` is useful when you have several tests for the same function or component.

Simple definition:

`describe()` creates a group of related tests.

Beginner rule:

Use `describe()` when a file has multiple tests for the same thing.

15. expect()

`expect()` starts an assertion.

An assertion checks whether something is true.

Example:

```
expect(2 + 2).toBe(4);
```

This assertion says:

The result of `2 + 2` should be `4`.

Another example:

```
expect(formatName("react")).toBe("React");
```

This assertion says:

The formatted name should be `"React"`.

Beginner rule:

`expect()` receives the actual value.

16. Matchers

A matcher is the method after `expect()`.

Example:

```
expect(2 + 2).toBe(4);
```

Here, `.toBe()` is the matcher.

Common matchers:

```
expect(2 + 2).toBe(4);  
expect(user).toEqual({ name: "Ana" });  
expect(["React", "Vue"]).toContain("React");  
expect(button).toBeInTheDocument();  
expect(alert).toHaveTextContent("Name is required");
```

16.1 `.toBe()`

Use `.toBe()` for simple exact values.

Good for:

- Numbers.
- Strings.
- Booleans.
- `null`.
- `undefined`.

Example:

```
expect(add(2, 3)).toBe(5);  
expect(formatName("react")).toBe("React");
```

16.2 `.toEqual()`

Use `.toEqual()` for objects and arrays.

Example:

```
expect(getUser()).toEqual({ name: "Ana", role: "Admin" });
```

Do not usually use `.toBe()` for different object or array instances.

Better:

```
expect([1, 2, 3]).toEqual([1, 2, 3]);
```

16.3 `.toContain()`

Use `.toContain()` when checking whether an array contains a value.

Example:

```
expect(["React", "Vue", "Angular"]).toContain("React");
```

16.4 `.toBeInTheDocument()`

Use `.toBeInTheDocument()` with React Testing Library to check whether an element exists on the page.

Example:

```
expect(screen.getByText("Submit")).toBeInTheDocument();
```

16.5 `.toHaveTextContent()`

Use `.toHaveTextContent()` to check visible text inside an element.

Example:

```
expect(screen.getByRole("alert")).toHaveTextContent("Name is required");
```

17. Passing Tests

A test passes when the actual result matches the expected result.

Example:

```
function add(a, b) {  
  return a + b;  
}  
  
test("adds two numbers", () => {  
  expect(add(2, 3)).toBe(5);  
});
```

Actual result:

```
5
```

Expected result:

```
5
```

The test passes.

Passing test means:

This checked behavior currently works.

18. Failing Tests

A test fails when the actual result does not match the expected result.

Broken function:

```
function add(a, b) {  
  return a - b;  
}
```

Test:

```
test("adds two numbers", () => {  
  expect(add(2, 3)).toBe(5);  
});
```

Actual result:

-1

Expected result:

5

The test fails.

A failing test is not always bad.

It gives you information.

It says:

This behavior does not match the expected behavior.

Beginner rule:

Read failing tests carefully. They usually tell you what expected value and received value were different.

19. Example: `formatName()` Utility Function

Create this file:

```
// src/utils/formatName.js

export function formatName(str = "") {
  if (!str) return "";
  return str.charAt(0).toUpperCase() + str.slice(1);
}
```

What this function should do:

Input	Output
"react"	"React"
"React"	"React"
""	""
no input	""

Explanation:

```
if (!str) return "";
```

If the input is empty, return an empty string.

```
str.charAt(0).toUpperCase()
```

Take the first character and make it uppercase.

```
str.slice(1)
```

Take the rest of the string after the first character.

```
return str.charAt(0).toUpperCase() + str.slice(1);
```

Join the uppercase first letter with the rest of the string.

20. Test: `formatName()`

Create this file:

```
// src/utils/formatName.test.js

import { describe, expect, test } from "vitest";
import { formatName } from "../formatName";

describe("formatName", () => {
  test("capitalizes lowercase input", () => {
    expect(formatName("react")).toBe("React");
  });

  test("returns empty string for no input", () => {
    expect(formatName()).toBe("");
  });

  test("keeps already-capitalized input correct", () => {
    expect(formatName("React")).toBe("React");
  });
});
```

20.1 Test 1

```
test("capitalizes lowercase input", () => {
  expect(formatName("react")).toBe("React");
});
```

Meaning:

If the input is `"react"`, the output should be `"React"`.

20.2 Test 2

```
test("returns empty string for no input", () => {  
  expect(formatName()).toBe("");  
});
```

Meaning:

If no input is provided, the function should not crash. It should return an empty string.

20.3 Test 3

```
test("keeps already-capitalized input correct", () => {  
  expect(formatName("React")).toBe("React");  
});
```

Meaning:

If the input is already capitalized, the output should still be correct.

Beginner rule:

Test the normal case and at least one edge case.

21. Edge Cases

An edge case is an unusual or boundary input.

Example normal case:

```
formatName("react")
```

Example edge cases:

```
formatName("")  
formatName()  
formatName("a")
```

Why edge cases matter:

- Users may submit empty fields.
- APIs may return missing data.
- Functions may receive unexpected values.
- Bugs often hide in unusual cases.

Beginner rule:

Do not only test the happy path. Test simple edge cases too.

22. Happy Path

The happy path is the normal successful case.

Example:

```
formatName("react")
```

Expected:

```
React
```

This is a happy path because everything is normal.

Happy path form example:

```
User enters valid name
↓
User enters valid email
↓
User submits form
↓
Signup succeeds
```

Beginner rule:

Start with the happy path, then add edge cases.

23. Test Files

A test file is a file that contains tests.

Common naming patterns:

```
formatName.test.js
Button.test.jsx
calculateCartTotal.test.js
Alert.test.jsx
```

A common file structure:

```
src/
  utils/
    formatName.js
    formatName.test.js
  components/
    Button.jsx
    Button.test.jsx
```

Simple rule:

Put the test near the code it tests, especially while learning.

24. Vitest Beginner Context

In Vite React projects, beginners often use Vitest for testing.

Typical imports:

```
import { describe, expect, test } from "vitest";
```

For React component tests, you may also use:

```
import { render, screen } from "@testing-library/react";
import "@testing-library/jest-dom/vitest";
```

What these are for:

Import	Purpose
<code>test</code>	Creates a test
<code>describe</code>	Groups tests
<code>expect</code>	Starts an assertion
<code>render</code>	Renders a React component in a test
<code>screen</code>	Finds elements on the rendered screen
<code>jest-dom</code> matchers	Adds matchers like <code>toBeInTheDocument()</code>

Beginner note:

You do not need to master test setup immediately. First understand what the test is checking.

25. Cart Utility Example

Create this file:

```
// src/utils/calculateCartTotal.js

export function calculateCartTotal(items = []) {
  return items.reduce((total, item) => {
    return total + item.price * item.quantity;
  }, 0);
}
```

What this function does:

For each cart item:
 price × quantity

Then add all item totals together.

Example:

```
calculateCartTotal([
  { price: 10, quantity: 2 },
  { price: 5, quantity: 3 },
]);
```

Calculation:

```
10 × 2 = 20  
5 × 3 = 15  
20 + 15 = 35
```

Expected result:

35

26. Test: `calculateCartTotal()`

Create this file:

```
// src/utils/calculateCartTotal.test.js  
  
import { describe, expect, test } from "vitest";  
import { calculateCartTotal } from "../calculateCartTotal";  
  
describe("calculateCartTotal", () => {  
  test("returns 0 for an empty cart", () => {  
    expect(calculateCartTotal([])).toBe(0);  
  });  
  
  test("calculates total for one item", () => {  
    expect(  
      calculateCartTotal([{ price: 10, quantity: 2 }])  
    ).toBe(20);  
  });  
  
  test("calculates total for multiple items", () => {  
    expect(  
      calculateCartTotal([  
        { price: 10, quantity: 2 },  
        { price: 5, quantity: 3 },  
      ])  
    ).toBe(35);  
  });  
});
```

This test file checks:

1. Empty cart.
2. One item.
3. Multiple items.

Why this is a good beginner test:

- It is small.
- It is clear.
- It checks real app logic.
- It does not need React rendering.

Beginner rule:

Utility functions are often the easiest place to start testing.

27. Assertions

An assertion is a check inside a test.

Example:

```
expect(calculateCartTotal([])).toBe(0);
```

The assertion checks this:

An empty cart should have a total of `0`.

A test can have one assertion or multiple assertions.

Simple example with one assertion:

```
test("returns 0 for empty cart", () => {  
  expect(calculateCartTotal([])).toBe(0);  
});
```

Example with multiple assertions:

```
test("formats different names", () => {  
  expect(formatName("react")).toBe("React");  
});
```



```
expect(formatName("vue")).toBe("Vue");
});
```

Beginner rule:

Keep tests focused. One clear behavior per test is usually easier to understand.

28. TDD: Test-Driven Development

TDD means Test-Driven Development.

TDD means you write the test before writing the final code.

The TDD cycle:

```
Write a test
↓
Run the test
↓
Watch it fail
↓
Write the minimum code to pass
↓
Run the test again
↓
Watch it pass
↓
Refactor if needed
↓
Repeat
```

This is often called:

```
Red → Green → Refactor
```

28.1 Red

Red means the test fails.

This is expected at first because the feature may not exist yet.

28.2 Green

Green means the test passes.

This means your code satisfies the test.

28.3 Refactor

Refactor means improving code structure without changing behavior.

The test should still pass after refactoring.

Beginner rule:

TDD helps you think about expected behavior before implementation details.

29. TDD Example

First, write the test:

```
// src/utils/calculateCartTotal.test.js

import { expect, test } from "vitest";
import { calculateCartTotal } from "../calculateCartTotal";

test("adds item prices", () => {
  expect(
    calculateCartTotal([
      { price: 10, quantity: 2 },
      { price: 5, quantity: 1 },
    ])
  ).toBe(25);
});
```

At this moment, `calculateCartTotal` may not exist or may be incomplete.

The test fails.

Then write the function:

```
// src/utils/calculateCartTotal.js
```

```
export function calculateCartTotal(items) {  
  return items.reduce((total, item) => {  
    return total + item.price * item.quantity;  
  }, 0);  
}
```

Now run the test again.

The test passes if the result is `25`.

TDD is not mandatory for every beginner task.

But it is useful because it asks:

What should this code do?

30. BDD: Behavior-Driven Development

BDD means Behavior-Driven Development.

BDD focuses on behavior from the user or business point of view.

Instead of starting with technical details, BDD starts with expected behavior.

Common BDD structure:

```
Given some starting situation  
When something happens  
Then some result should happen
```

Simple definition:

BDD describes what the user expects from the system.

31. BDD Example

Login behavior:

```
Given the user enters a valid email and password
When the user submits the login form
Then the dashboard should appear
```

Cart behavior:

```
Given the cart has two products
When the cart total is calculated
Then the total should include price times quantity for each product
```

Form validation behavior:

```
Given the name field is empty
When the user submits the form
Then an error message should appear
```

BDD is useful because it connects tests to real user needs.

Beginner rule:

BDD helps you describe behavior before thinking about code structure.

32. TDD vs BDD

Topic	TDD	BDD
Full name	Test-Driven Development	Behavior-Driven Development
Focus	Write tests before code	Describe user/business behavior
Language	Technical test cases	Given/When/Then behavior
Useful for	Building small units carefully	Aligning code with expected behavior

Simple comparison:

```
TDD asks: What should this function return?
BDD asks: What should the user experience?
```

Both can work together.

33. Mocks, Stubs, and Fakes

Mocks, stubs, and fakes are replacements used in tests.

They help you avoid using real external systems during a test.

External systems may include:

- APIs.
- Backends.
- Databases.
- File systems.
- Authentication services.
- Payment services.

Why replacements are useful:

- Real APIs can be slow.
- Real APIs can fail.
- Real databases can change.
- Tests should be predictable.
- Tests should not depend on external services unless necessary.

Simple definition:

Mocks, stubs, and fakes are controlled test replacements.

34. Mock

A mock is a fake version of something used to simulate data or check interaction.

Simple definition:

A mock pretends to be a real thing during a test.

Example mock API data:

```
const mockPosts = [  
  { id: 1, title: "Learn React" },  
  { id: 2, title: "Write Tests" },  
];
```

This replaces real API data.

Instead of calling a real API, your test uses known data.

Example:

```
expect(getPostTitles(mockPosts)).toEqual([
  "Learn React",
  "Write Tests",
]);
```

Beginner rule:

Use mock data when you want predictable test input.

35. Stub

A stub returns a predefined answer.

Simple definition:

A stub is a replacement that gives a fixed response.

Example:

```
function getUserStub() {
  return { id: 1, name: "Sangeeth" };
}
```

This function does not fetch a real user.

It always returns the same user.

Example use:

```
const user = getUserStub();
expect(user.name).toBe("Sangeeth");
```

Beginner rule:

Use a stub when you need a known response without real work.

36. Fake

A fake is a simplified working replacement.

Simple definition:

A fake works like the real thing, but in a smaller and simpler way.

Example fake in-memory database:

```
const fakeDb = {
  users: [],
  addUser(user) {
    this.users.push(user);
  },
  getUsers() {
    return this.users;
  },
};
```

This fake database does not use a real database server.

It stores data in an array.

Example use:

```
fakeDb.addUser({ id: 1, name: "Ana" });
expect(fakeDb.getUsers()).toEqual([{ id: 1, name: "Ana" }]);
```

Beginner rule:

Use a fake when you need a lightweight working version of a real dependency.

37. Mock vs Stub vs Fake

Type	Simple Meaning	Example
Mock	Pretends to be something and may check interaction	Mock API posts

Type	Simple Meaning	Example
Stub	Returns a fixed response	<code>getUserStub()</code> always returns same user
Fake	A simple working replacement	Fake in-memory database

Memory rule:

```
Mock = pretend object/data
Stub = fixed answer
Fake = simple working version
```

For Sprint 14, do not overcomplicate these terms.

The main idea is:

Tests often use controlled replacements instead of real external systems.

38. Mock API Data Practice

Function:

```
// src/utils/getPostTitles.js

export function getPostTitles(posts = []) {
  return posts.map((post) => post.title);
}
```

Test:

```
// src/utils/getPostTitles.test.js

import { expect, test } from "vitest";
import { getPostTitles } from "../getPostTitles";

test("extracts titles from mocked posts", () => {
  const mockPosts = [
    { id: 1, title: "Learn React" },
    { id: 2, title: "Write Tests" },
  ];
});
```



```
expect(getPostTitles(mockPosts)).toEqual([
  "Learn React",
  "Write Tests",
]);
});
```

What this test checks:

```
Input: array of post objects
↓
Function maps posts to titles
↓
Output: array of post titles
```

Important point:

`mockPosts` replaces real API data.

The test is predictable because the data is controlled.

39. React Testing Library

React Testing Library is used to test React components.

Simple definition:

React Testing Library helps you test what users can see and do.

It encourages this mindset:

Test the component like a user would use it.

This means you should usually test:

- Visible text.
- Buttons.
- Links.
- Labels.
- Inputs.
- Alerts.
- Form behavior.
- Conditional messages.

You should avoid testing internal implementation details too early.

Bad mindset:

Is the internal state variable `isOpen` true?

Better mindset:

Is the menu visible to the user?

Beginner rule:

Test visible behavior, not private implementation details.

40. `render()`

`render()` renders a React component in the test environment.

Example:

```
render(<Button label="Submit" onClick={() => {}} />);
```

This means:

Show this component inside the test so we can inspect it.

`render()` comes from React Testing Library:

```
import { render, screen } from "@testing-library/react";
```

Beginner rule:

Use `render()` before checking what a component displays.

41. `screen`

`screen` helps you find elements that were rendered.

Example:

```
screen.getByText("Submit")
```

This finds text on the screen.

Example:

```
screen.getByRole("button", { name: "Submit" })
```

This finds a button with the accessible name `Submit`.

Example:

```
screen.getByLabelText("Email")
```

This finds an input connected to a label that says `Email`.

Beginner rule:

`screen` lets your test inspect the rendered component like a user would.

42. Common React Testing Library Queries

42.1 `getByText()`

Use `getByText()` to find visible text.

Example:

```
expect(screen.getByText("Welcome")).toBeInTheDocument();
```

Meaning:

The text `Welcome` should be visible.

42.2 `getByRole()`

Use `getByRole()` to find an element by its accessible role.

Example:

```
expect(screen.getByRole("button", { name: "Submit" })).toBeInTheDocument();
```

Meaning:

There should be a button named `Submit`.

Common roles:

- `button`
- `link`
- `textbox`
- `heading`
- `alert`

`getByRole()` is often a strong query because it matches how assistive technologies understand the page.

42.3 `getByLabelText()`

Use `getByLabelText()` for form inputs connected to labels.

Example component:

```
<label htmlFor="email">Email</label>  
<input id="email" type="email" />
```

Test:

```
expect(screen.getByLabelText("Email")).toBeInTheDocument();
```

This is good because real users see the label.

42.4 `queryByRole()`

Use `queryByRole()` when an element might not exist.

Example:

```
expect(screen.queryByRole("alert")).not.toBeInTheDocument();
```

Use `queryBy...` when checking absence.

Beginner query rule:

Element should exist → use getBy...
Element should not exist → use queryBy...

43. Component Test Example: Button

Component:

```
// src/components/Button.jsx

export default function Button({ label, onClick }) {
  return (
    <button type="button" onClick={onClick}>
      {label}
    </button>
  );
}
```

What this component does:

- Receives a `label` prop.
- Receives an `onClick` prop.
- Renders a real button.
- Shows the label inside the button.

Test:

```
// src/components/Button.test.jsx

import { render, screen } from "@testing-library/react";
import { expect, test } from "vitest";
import "@testing-library/jest-dom/vitest";
import Button from "../Button";

test("shows the button label", () => {
  render(<Button label="Submit" onClick={() => {}} />);

  expect(screen.getByRole("button", { name: "Submit" })).toBeInTheDocument();
});
```

Explanation:

```
render(<Button label="Submit" onClick={() => {}} />);
```

This renders the button.

```
screen.getByRole("button", { name: "Submit" })
```

This searches for a button named `Submit`.

```
.toBeInTheDocument()
```

This checks that the button exists.

Why this is a good test:

- It checks visible behavior.
- It uses an accessible query.
- It does not check private component internals.

44. Component Test Example: Alert

Component:

```
// src/components/Alert.jsx

export default function Alert({ type = "info", message }) {
  if (!message) return null;

  return (
    <div role="alert">
      {type}: {message}
    </div>
  );
}
```

What this component does:

- Receives a `type` prop.
- Receives a `message` prop.

- If there is no message, it renders nothing.
- If there is a message, it renders an alert.

Test:

```
// src/components/Alert.test.jsx

import { render, screen } from "@testing-library/react";
import { expect, test } from "vitest";
import "@testing-library/jest-dom/vitest";
import Alert from "../Alert";

test("shows an alert message", () => {
  render(<Alert type="error" message="Name is required" />);

  expect(screen.getByRole("alert")).toHaveTextContent(
    "error: Name is required"
  );
});

test("renders nothing when message is missing", () => {
  render(<Alert />);

  expect(screen.queryByRole("alert")).not.toBeInTheDocument();
});
```

44.1 First Test

```
test("shows an alert message", () => {
  render(<Alert type="error" message="Name is required" />);

  expect(screen.getByRole("alert")).toHaveTextContent(
    "error: Name is required"
  );
});
```

Meaning:

If the component receives an error message, the alert should show that message.

44.2 Second Test

```
test("renders nothing when message is missing", () => {
  render(<Alert />);
```

```
expect(screen.queryByRole("alert")).not.toBeInTheDocument();
});
```

Meaning:

If there is no message, there should be no alert on the screen.

Beginner rule:

Test both the visible case and the hidden case for conditional UI.

45. Simple Form Component Test Example

Component:

```
// src/components/SignupForm.jsx

export default function SignupForm() {
  return (
    <form>
      <label htmlFor="name">Name</label>
      <input id="name" type="text" />

      <button type="submit">Sign up</button>
    </form>
  );
}
```

Test:

```
// src/components/SignupForm.test.jsx

import { render, screen } from "@testing-library/react";
import { expect, test } from "vitest";
import "@testing-library/jest-dom/vitest";
import SignupForm from "../SignupForm";

test("renders name input and submit button", () => {
  render(<SignupForm />);

  expect(screen.getByLabelText("Name")).toBeInTheDocument();
});
```



```
expect(screen.getByRole("button", { name: "Sign up" })).toBeInTheDocument();
});
```

Why this test is good:

- It checks the input through its label.
- It checks the button through its role and name.
- It encourages accessible JSX.

Beginner rule:

Good accessibility often makes components easier to test.

46. Testing Conditional Text

Component:

```
// src/components/StatusMessage.jsx

export default function StatusMessage({ isLoggedIn }) {
  return (
    <p>{isLoggedIn ? "Welcome back" : "Please log in"}</p>
  );
}
```

Test:

```
// src/components/StatusMessage.test.jsx

import { render, screen } from "@testing-library/react";
import { expect, test } from "vitest";
import "@testing-library/jest-dom/vitest";
import StatusMessage from "../StatusMessage";

test("shows welcome message when logged in", () => {
  render(<StatusMessage isLoggedIn={true} />);

  expect(screen.getByText("Welcome back")).toBeInTheDocument();
});

test("shows login message when logged out", () => {
  render(<StatusMessage isLoggedIn={false} />);
```

```
    expect(screen.getByText("Please log in")).toBeInTheDocument();
  });
```

This is useful because conditional rendering is common in React.

Beginner rule:

If a component has two visible states, test both states.

47. Testing Props

Props change what a component displays.

Component:

```
// src/components/UserCard.jsx

export default function UserCard({ name, role }) {
  return (
    <article>
      <h2>{name}</h2>
      <p>{role}</p>
    </article>
  );
}
```

Test:

```
// src/components/UserCard.test.jsx

import { render, screen } from "@testing-library/react";
import { expect, test } from "vitest";
import "@testing-library/jest-dom/vitest";
import UserCard from "../UserCard";

test("shows user name and role", () => {
  render(<UserCard name="Ana" role="Frontend Developer" />);

  expect(screen.getByText("Ana")).toBeInTheDocument();
  expect(screen.getByText("Frontend Developer")).toBeInTheDocument();
});
```

What this test checks:

The component displays the props it receives.

Beginner rule:

If props control visible output, test the visible output.

48. What Not to Test Yet

As a beginner, do not start with the hardest testing topics.

Avoid starting with:

- Complex API mocking.
- Full authentication flows.
- Full checkout flows.
- Testing internal React state directly.
- Testing private implementation details.
- Snapshot tests everywhere.
- Large end-to-end testing frameworks before you understand unit and component tests.

Start with:

- Pure utility functions.
- Visible text.
- Buttons.
- Labels.
- Inputs.
- Simple conditional messages.
- Simple form validation messages.
- Small component behavior.

Beginner rule:

Test small, clear behavior first.

49. Implementation Details

Implementation details are internal code choices that users do not directly see.

Examples:

- A state variable name.
- A helper function inside a component.
- Whether you use `useState` or `useReducer`.
- The exact component structure inside a component.

Bad beginner testing style:

```
expect(component.state.isOpen).toBe(true);
```

Better testing style:

```
expect(screen.getByText("Menu")).toBeInTheDocument();
```

Why?

The user does not care whether your internal state is called `isOpen`.

The user cares whether the menu appears.

Beginner rule:

Test what the user can observe, not how the component is built internally.

50. Tests as Documentation

Tests can act like documentation.

Example:

```
test("returns 0 for an empty cart", () => {  
  expect(calculateCartTotal([])).toBe(0);  
});
```

This tells another developer:

An empty cart total should be 0.

Good test names explain behavior.

Poor test name:

```
test("cart", () => {});
```

Better test name:

```
test("returns 0 for an empty cart", () => {});
```

Beginner rule:

A good test explains what the code is supposed to do.

51. Test-First Thinking Without Full TDD

You do not always need strict TDD.

But you should practice test-first thinking.

Ask:

```
What is the input?  
What should the output be?  
What should the user see?  
What should happen when the value is empty?  
What should happen when the data is missing?
```

Example for `formatName()`:

```
Input: "react"  
Expected output: "React"
```

Example for `Alert`:

```
Input: message = "Name is required"  
Expected UI: alert appears with that message
```

Beginner rule:

Clear expectations make testing easier.

52. Common Beginner Testing Mistakes

52.1 Bad Test Names

Bad:

```
test("works", () => {});
```

Better:

```
test("capitalizes lowercase input", () => {});
```

52.2 Testing Too Much at Once

Bad:

```
test("entire app works", () => {});
```

Better:

```
test("shows error message when name is empty", () => {});
```

52.3 Testing Implementation Details

Bad:

```
expect(internalState).toBe(true);
```

Better:

```
expect(screen.getByText("Open")).toBeInTheDocument();
```

52.4 Forgetting Edge Cases

Only testing this:

```
formatName("react")
```

But forgetting:

```
formatName("")  
formatName()
```

52.5 Using Real API Data Too Early

Bad beginner approach:

```
// Test depends on a real external API
```

Better beginner approach:

```
const mockPosts = [  
  { id: 1, title: "Learn React" },  
];
```

Beginner rule:

Keep beginner tests small, predictable, and focused.

53. What Makes a Good Beginner Test?

A good beginner test is:

- Small.
- Clear.
- Focused on one behavior.
- Named well.
- Predictable.
- Not dependent on a real API.
- Easy to understand when it fails.

Good example:

```
test("returns 0 for an empty cart", () => {  
  expect(calculateCartTotal([])).toBe(0);  
});
```

Why it is good:

- The name is clear.
- The input is clear.
- The expected output is clear.
- The behavior is small.

54. Sprint 14 Practice Block

54.1 Warm-Up Drill 1: Basic Test

Write this test:

```
test("adds numbers", () => {  
  expect(2 + 2).toBe(4);  
});
```

Explain it in simple words:

I expect `2 + 2` to equal `4`.

54.2 Warm-Up Drill 2: Test Description

Write better names for these weak test names:

Bad:

```
test("works", () => {});
```

Better:

```
test("returns empty string when no name is provided", () => {});
```

Bad:


```
test("button", () => {});
```

Better:

```
test("shows the submit button", () => {});
```

54.3 Warm-Up Drill 3: Matcher Choice

Choose the better matcher:

```
expect(5).toBe(5);
```

Good for numbers.

```
expect(["React", "Vue"]).toEqual(["React", "Vue"]);
```

Good for arrays.

55. Guided Coding Task 1: `formatName()`

Create:

```
// src/utils/formatName.js

export function formatName(str = "") {
  if (!str) return "";
  return str.charAt(0).toUpperCase() + str.slice(1);
}
```

Create:

```
// src/utils/formatName.test.js

import { describe, expect, test } from "vitest";
import { formatName } from "../formatName";

describe("formatName", () => {
  test("capitalizes lowercase input", () => {
```

```

    expect(formatName("react")).toBe("React");
  });

  test("returns empty string for empty input", () => {
    expect(formatName("")).toBe("");
  });

  test("returns empty string when no input is provided", () => {
    expect(formatName()).toBe("");
  });

  test("keeps already-capitalized input correct", () => {
    expect(formatName("React")).toBe("React");
  });
});

```

Self-check:

- Does lowercase input work?
- Does empty input work?
- Does missing input work?
- Does already-capitalized input work?

56. Guided Coding Task 2: Cart Total

Create:

```

// src/utils/calculateCartTotal.js

export function calculateCartTotal(items = []) {
  return items.reduce((total, item) => {
    return total + item.price * item.quantity;
  }, 0);
}

```

Create:

```

// src/utils/calculateCartTotal.test.js

import { describe, expect, test } from "vitest";
import { calculateCartTotal } from "../calculateCartTotal";

describe("calculateCartTotal", () => {

```

```

test("returns 0 for an empty cart", () => {
  expect(calculateCartTotal([])).toBe(0);
});

test("calculates total for one item", () => {
  expect(
    calculateCartTotal([{ price: 10, quantity: 2 }])
  ).toBe(20);
});

test("calculates total for multiple items", () => {
  expect(
    calculateCartTotal([
      { price: 10, quantity: 2 },
      { price: 5, quantity: 3 },
    ])
  ).toBe(35);
});

```

Self-check:

- Empty cart returns `0`.
- One item calculates correctly.
- Multiple items calculate correctly.

57. Guided Coding Task 3: Button Component Test

Create:

```

// src/components/Button.jsx

export default function Button({ label, onClick }) {
  return (
    <button type="button" onClick={onClick}>
      {label}
    </button>
  );
}

```

Create:

```
// src/components/Button.test.jsx

import { render, screen } from "@testing-library/react";
import { expect, test } from "vitest";
import "@testing-library/jest-dom/vitest";
import Button from "../Button";

test("shows the button label", () => {
  render(<Button label="Submit" onClick={() => {}} />);

  expect(screen.getByRole("button", { name: "Submit" })).toBeInTheDocument();
});
```

Self-check:

- The component renders a real button.
- The button has visible text.
- The test checks the button like a user would.

58. Guided Coding Task 4: Alert Component Test

Create:

```
// src/components/Alert.jsx

export default function Alert({ type = "info", message }) {
  if (!message) return null;

  return (
    <div role="alert">
      {type}: {message}
    </div>
  );
}
```

Create:

```
// src/components/Alert.test.jsx

import { render, screen } from "@testing-library/react";
import { expect, test } from "vitest";
import "@testing-library/jest-dom/vitest";
```

```
import Alert from './Alert';

test("shows an alert message", () => {
  render(<Alert type="error" message="Name is required" />);

  expect(screen.getByRole("alert")).toHaveTextContent(
    "error: Name is required"
  );
});

test("renders nothing when message is missing", () => {
  render(<Alert />);

  expect(screen.queryByRole("alert")).not.toBeInTheDocument();
});
```

Self-check:

- Alert appears when message exists.
- Alert does not appear when message is missing.
- `getByRole()` is used for expected presence.
- `queryByRole()` is used for expected absence.

59. Debug This Test

Broken code:

```
test("works", () => {
  expect(formatName("react")).toBe("react");
});
```

Problems:

1. Test name is unclear.
2. Expected value is wrong if the function should capitalize the name.

Better:

```
test("capitalizes lowercase input", () => {
  expect(formatName("react")).toBe("React");
});
```

60. Debug This Component Test

Broken test:

```
test("button", () => {  
  render(<Button label="Submit" />);  
  
  expect(screen.getByText("Cancel")).toBeInTheDocument();  
});
```

Problems:

1. Test name is unclear.
2. Component receives label "Submit".
3. Test searches for "Cancel".
4. If `onClick` is required by the component, it should be provided.

Better:

```
test("shows the button label", () => {  
  render(<Button label="Submit" onClick={() => {}} />);  
  
  expect(screen.getByRole("button", { name: "Submit" })).toBeInTheDocument();  
});
```

61. Build Something Small: Testing Mini Project

Build a small project with these files:

```
src/  
  components/  
    Button.jsx  
    Button.test.jsx  
    Alert.jsx  
    Alert.test.jsx  
  utils/  
    formatName.js  
    formatName.test.js  
    calculateCartTotal.js  
    calculateCartTotal.test.js
```

Your goal:

- Test one formatting utility.
- Test one cart utility.
- Test one button component.
- Test one alert component.

Expected result:

- Utility tests pass.
- Component tests pass.
- Test names clearly describe behavior.
- Component tests check visible output.

62. Challenge Task: Simple Signup Form Test

Component:

```
// src/components/SignupForm.jsx

import { useState } from "react";

export default function SignupForm() {
  const [name, setName] = useState("");
  const [submitted, setSubmitted] = useState(false);

  const nameIsValid = name.trim().length > 0;

  function handleSubmit(event) {
    event.preventDefault();
    setSubmitted(true);

    if (!nameIsValid) {
      return;
    }

    console.log("Submitted:", name);
  }

  return (
    <form onSubmit={handleSubmit}>
      <label htmlFor="name">Name</label>
      <input
        id="name"
        value={name}
      />
    </form>
  );
}
```

```

        onChange={(event) => setName(event.target.value)}
      />

      {submitted && !nameIsValid && (
        <p role="alert">Name is required</p>
      )}

      <button type="submit">Sign up</button>
    </form>
  );
}

```

Beginner test for initial render:

```

// src/components/SignupForm.test.jsx

import { render, screen } from "@testing-library/react";
import { expect, test } from "vitest";
import "@testing-library/jest-dom/vitest";
import SignupForm from "../SignupForm";

test("renders name input and sign up button", () => {
  render(<SignupForm />);

  expect(screen.getByLabelText("Name")).toBeInTheDocument();
  expect(screen.getByRole("button", { name: "Sign up" })).toBeInTheDocument();
});

```

This is enough for Sprint 14 if you are still new.

Later, you can test typing and submitting with `userEvent`, but Sprint 14 focuses on the testing foundation first.

63. Revision Questions

Answer these without looking at the notes.

1. What is testing?
2. Why is manual testing alone not enough?
3. What is automated testing?
4. What does SDLC mean?
5. What are the main SDLC stages?
6. What is a unit test?

7. What does `test()` do?
 8. What does `expect()` do?
 9. What is a matcher?
 10. When should you use `.toBe()`?
 11. When should you use `.toEqual()`?
 12. What is an assertion?
 13. What is a passing test?
 14. What is a failing test?
 15. What is the TDD cycle?
 16. What does Red → Green → Refactor mean?
 17. What is BDD?
 18. What does Given/When/Then mean?
 19. What is a mock?
 20. What is a stub?
 21. What is a fake?
 22. What does React Testing Library help you test?
 23. Why should you test visible behavior instead of implementation details?
 24. What does `render()` do?
 25. What does `screen` do?
 26. When should you use `getByRole()`?
 27. When should you use `getByLabelText()`?
 28. When should you use `queryByRole()`?
 29. What should beginners avoid testing too early?
 30. What makes a good beginner test?
-

64. Sprint 14 Checklist

You are ready to move on from Sprint 14 when you can:

- Explain manual testing.
- Explain automated testing.
- Explain why testing matters.
- List the SDLC stages.
- Write a basic `test()` block.
- Use `expect()` with `.toBe()`.
- Use `.toEqual()` for arrays or objects.
- Write tests for `formatName()`.
- Write tests for `calculateCartTotal()`.
- Explain passing and failing tests.
- Explain TDD in simple words.
- Explain BDD in simple words.
- Compare mocks, stubs, and fakes.
- Render a simple React component in a test.
- Use `screen.getByText()`.
- Use `screen.getByRole()`.

- Use `screen.getByLabelText()`.
- Use `queryByRole()` when checking that something is absent.
- Avoid testing implementation details.
- Write clear test names.

65. Sprint 14 One-Page Memory Summary

Testing means checking whether code behaves correctly.

Manual testing means a human checks the app by using it.

Automated testing means test code checks behavior for you.

SDLC means Software Development Life Cycle:

Planning → Design → Implementation → Testing → Deployment → Maintenance

A unit test checks one small piece of code.

Basic test shape:

```
test("behavior description", () => {  
  expect(actualValue).toBe(expectedValue);  
});
```

`test()` creates a test.

`describe()` groups tests.

`expect()` checks an actual value.

A matcher says what the expected value should be.

Examples:

```
expect(5).toBe(5);  
expect([1, 2]).toEqual([1, 2]);  
expect(["React"]).toContain("React");  
expect(button).toBeInTheDocument();
```

TDD means:

```
Write test → Fail → Write code → Pass → Refactor
```

BDD means describing behavior like this:

```
Given → When → Then
```

Mocks, stubs, and fakes are controlled test replacements.

```
Mock = pretend object/data  
Stub = fixed answer  
Fake = simple working version
```

React Testing Library helps test visible React behavior.

Good component tests ask:

What can the user see or do?

Common React Testing Library queries:

```
screen.getByText()  
screen.getByRole()  
screen.getByLabelText()  
screen.queryByRole()
```

Beginner testing rule:

Start with small utility functions and visible component behavior. Do not start by testing complex internals.

66. Final Sprint 14 Mental Model

Use this mental model:

```
I know what the code should do.  
↓  
I write a test that describes that behavior.  
↓  
The test runs the code.
```

↓

The test compares actual result with expected result.

↓

If they match, the test passes.

↓

If they do not match, the test fails and points me to the problem.

Sprint 14 is the point where you stop only asking:

Does my app work right now?

And start asking:

Can I prove that this important behavior still works after changes?